CO3 AT2

Experiments – Actor-Critic Methods (A2C, A3C)

Name: P. Hithaishi

Reg No: 192425196

Course Code: MLA0305

Course Name: Reinforcement Learning

---

1. Aim

To implement and analyze Actor-Critic reinforcement learning methods, namely Advantage Actor-Critic (A2C) and Asynchronous Advantage Actor-Critic (A3C), and compare their learning performance using reward and episode graphs.

2. Objective

- Implement the Actor-Critic architecture.
- Implement **A2C** for policy learning.
- Implement **A3C** using multiple parallel agents.
- Compare A2C and A3C based on cumulative reward and convergence.
- Visualize the training performance.

3. Basic Concept

Actor-Critic has two components:

Actor

The **Actor** selects an action according to the current policy.

Critic

The **Critic** estimates how good the current state is.

Advantage

The advantage tells us whether an action performed better or worse than expected.

where:

- = observed/estimated return

- = value predicted by Critic
- = advantage

4. A2C vs A3C

Feature	A2C	A3C
Full form	Advantage Actor-Critic	Asynchronous Advantage Actor-Critic
Agents	Multiple environments	Multiple workers
Updates	Synchronous	Asynchronous
Policy	Actor	Actor
Value estimation	Critic	Critic
Training	More stable	Faster exploration
Parallelism	Synchronous	Asynchronous

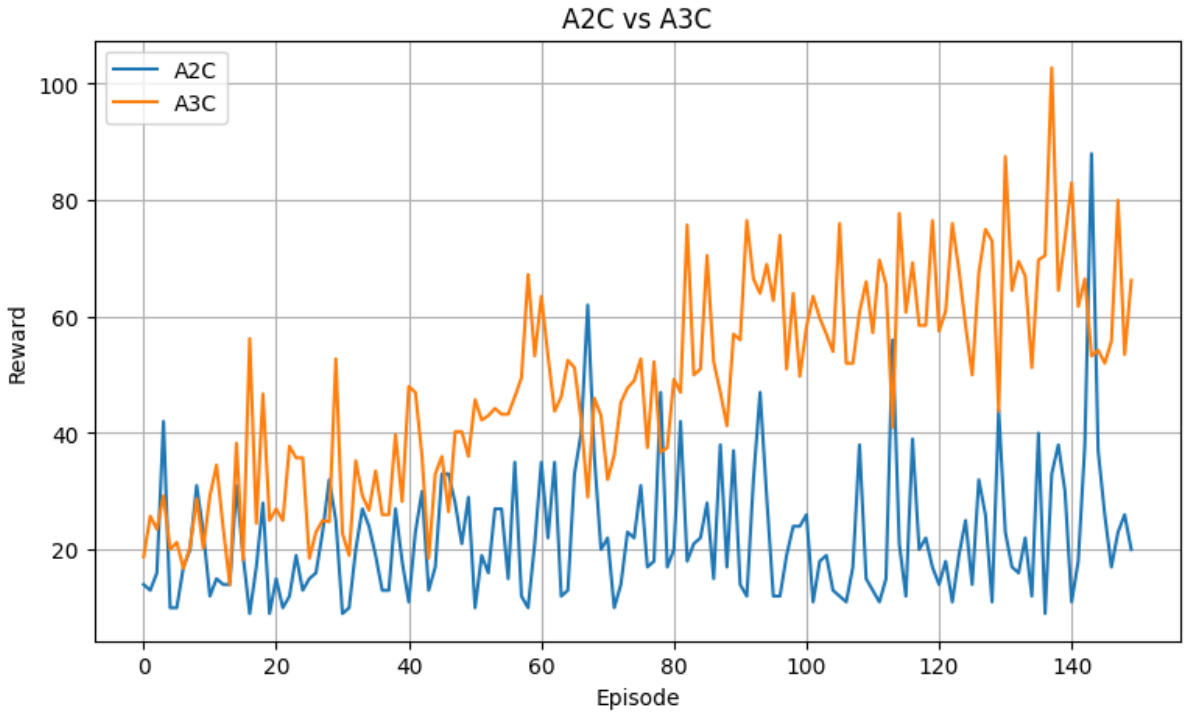
CODE:

```
import gymnasium as gym
import torch
import torch.nn as nn
import torch.optim as optim
import numpy as np
import matplotlib.pyplot as plt
env = gym.make("CartPole-v1")
state_n = env.observation_space.shape[0]
action_n = env.action_space.n
class AC(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc = nn.Linear(state_n, 64)
        self.actor = nn.Linear(64, action_n)
        self.critic = nn.Linear(64, 1)
```

```
    def forward(self, x):
        x = torch.relu(self.fc(x))
        return torch.softmax(self.actor(x), -1), self.critic(x)
def train(episodes=150, workers=1):
    model = AC()
    opt = optim.Adam(model.parameters(), lr=0.001)
    rewards = []
    for ep in range(episodes):
        total = []
        for w in range(workers):
            e = gym.make("CartPole-v1")
            s, _ = e.reset()
            lp, val, r = [], [], []
            done = False
            while not done:
                st = torch.tensor(s, dtype=torch.float32)
                p, v = model(st)
                d = torch.distributions.Categorical(p)
                a = d.sample()
                ns, reward, term, trunc, _ = e.step(a.item())
                done = term or trunc
                lp.append(d.log_prob(a))
                val.append(v.squeeze())
                r.append(reward)
                s = ns
            G, returns = 0, []
            for x in r[::-1]:
                G = x + 0.99 * G
                returns.insert(0, G)
```

```
            returns = torch.tensor(returns)
            val = torch.stack(val)
            lp = torch.stack(lp)
            adv = returns - val.detach()
            loss = -(lp * adv).mean() + 0.5 * (returns - val).pow(2).mean()
            opt.zero_grad()
            loss.backward()
            opt.step()
            total.append(sum(r))
            e.close()
        rewards.append(np.mean(total))
    return rewards
a2c = train(150, workers=1)
a3c = train(150, workers=4)
plt.figure(figsize=(9,5))
plt.plot(a2c, label="A2C")
plt.plot(a3c, label="A3C")
plt.xlabel("Episode")
plt.ylabel("Reward")
plt.title("A2C vs A3C")
plt.legend()
plt.grid()
plt.show()
```

OUTPUT:


A2C vs A3C
A2C
A3C
Reward
Episode
100
80
60
40
20
0
20
40
60
80
100
120
140